TASK 8 – Trace the Compilation Process

1. Stages of Compilation

The C compilation process converts source code into an executable program through several stages.

1. **Preprocessing (gcc -E):** Processes header files and macros and produces preprocessed source code.
2. **Compilation (gcc -S):** Converts the preprocessed C program into assembly code.
3. **Assembly (gcc -c):** Converts assembly code into an object file containing machine code.
4. **Linking:** Combines object files and required libraries to create the executable program.

2. Commands Used

For a source file named task08.c, the commands are:

gcc -E task08.c -o task08.i

gcc -S task08.c -o task08.s

gcc -c task08.c -o task08.o

gcc task08.o -o task08.exe

The files produced are:

Command	Output file	Purpose
gcc -E	task08.i	Preprocessed source
gcc -S	task08.s	Assembly code
gcc -c	task08.o	Object file
Linking	task08.exe	Executable program

3. What the Preprocessor Does

The preprocessor processes directives before compilation.

- #include inserts the contents of header files.
- #define replaces macros with their defined values.
- #ifdef DEBUG includes debugging code only when DEBUG is defined.

- Comments are removed from the preprocessed output.

For example, #define RATE 225 replaces occurrences of RATE with 225 during preprocessing.

4. Types of Errors

Error type	Meaning	Example
Compile-time error	The compiler cannot compile invalid C syntax or types	Missing semicolon after a statement
Link-time error	The linker cannot find a required function definition	Calling compute_bill() without linking its implementation
Run-time error	An error occurs while the program is executing	Dividing an integer by zero

These errors can be identified by checking the compiler, linker and program output.

5. Warnings Using -Wall -Wextra

The command:

gcc -Wall -Wextra task08.c -o task08.exe

enables additional compiler warnings. Warnings may identify unused variables, missing declarations, or suspicious code. They help improve code quality even when compilation succeeds.

I resolved warnings by removing unused variables, using correct format specifiers, and ensuring that functions and variables were declared correctly.

6. Alternative: Separate Source Files

A larger program can be divided into input.c, tariff.c and report.c, with shared declarations stored in a header file such as billing.h.

Each source file can be compiled separately:

gcc -Wall -Wextra -c input.c -o input.o

gcc -Wall -Wextra -c tariff.c -o tariff.o

gcc -Wall -Wextra -c report.c -o report.o

gcc input.o tariff.o report.o -o billing.exe

The linker combines the object files into one executable. All required object files must be included.

7. Compile-Time and Space Analysis

Factor	Single source file	Multiple source files
Initial build	Compiles the entire source file	Compiles each source file and links the object files
Change to one file	Usually recompiles the entire source file	Normally recompiles only the changed source file, then links again
Build time after a small change	Can be higher for a large program	Often lower when only one small module changes
Object-file storage	One object file	Multiple object files
Linking	Links one object file and libraries	Links all required object files and libraries

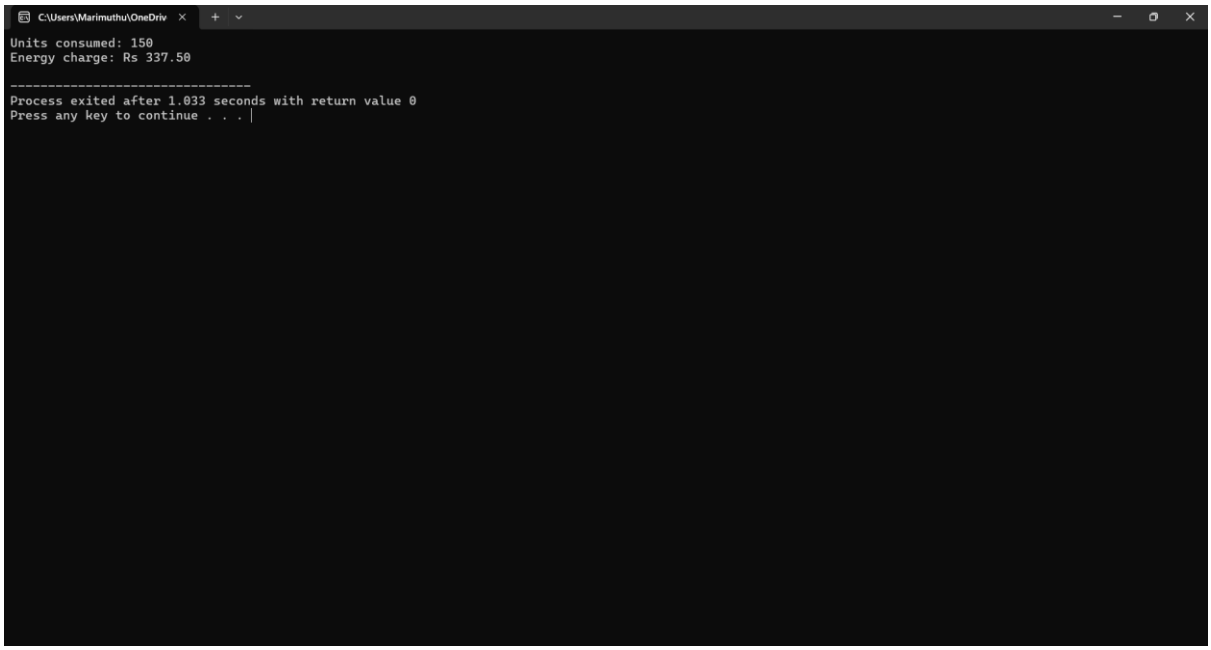
If the changed source file is small, separate compilation can save time because unchanged modules do not need recompilation. The object files occupy disk space until removed; their total size depends on the compiled code and debugging information.

Code :

```
#include <stdio.h>
#define RATE_PAISE 225
int main(void)
{
   int units = 150;
   long charge = (long)units * RATE_PAISE;
#ifdef DEBUG
   printf("[DEBUG] Units = %d\n", units);
   printf("[DEBUG] Rate = %d paise\n", RATE_PAISE);
   printf("[DEBUG] Charge = %ld paise\n", charge);
#endif
   printf("Units consumed: %d\n", units);
```

```
    printf("Energy charge: Rs %ld.%02ld\n",
        charge / 100, charge % 100);A
    return 0;
}
```

Output:



Debug Command:

gcc -E task08.c -o task08.i

gcc -S task08.c -o task08.s

gcc -c task08.c -o task08.o

gcc task08.o -o task08.exe

task08.exe

Debug output:

gcc -DDEBUG task08.c -o task08_debug.exe

task08_debug.exe

Conclusion

Understanding preprocessing, compilation, assembly and linking helps identify errors at the correct stage. Compiler warnings and separate source files improve the reliability and maintainability of C programs.